INSTITUTO TECNOLÓGICO DE TIJUANA

Patrones de Diseño

Decorator & Bridge

Proyecto: Unidad 3 — Patrones de Diseño de Software

Sistema: Aurum Grand Hotel — Cotizador de Habitaciones

Tecnologías: NestJS · React · TypeScript · EmailJS

Fecha: Abril 2026

PATRÓN 01

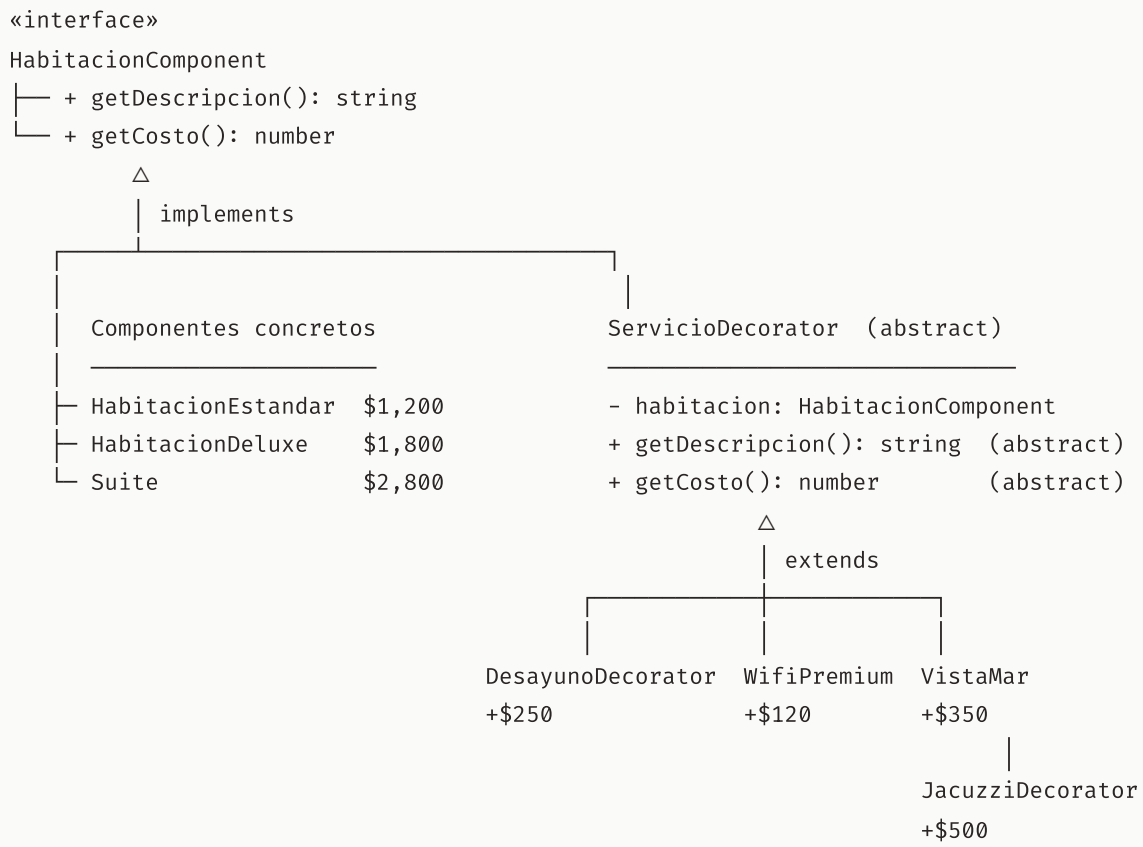
Decorator

Agregar responsabilidades a un objeto sin modificar su clase

¿Qué es el patrón Decorator?

El patrón Decorator permite añadir funcionalidades adicionales a un objeto de forma dinámica, sin alterar la clase original ni afectar a otros objetos del mismo tipo. Es una alternativa flexible a la herencia: en lugar de crear subclases para cada combinación posible de características, se envuelven los objetos en capas de decoradores que van sumando comportamiento.

Diagrama de clases



Clases participantes

INTERFACE **HabitacionComponent**

Componente abstracto. Define el contrato común que comparten tanto las habitaciones base como todos los decoradores. Garantiza que cualquier combinación sea intercambiable sin romper el código cliente.

`getDescripcion(): string` `getCosto(): number`

COMPONENTE CONCRETO **HabitacionEstandar · HabitacionDeluxe · Suite**

Implementan `HabitacionComponent` directamente. Son los objetos base sobre los que se aplican los decoradores. Cada uno define un precio fijo y una descripción simple. No contienen lógica de decoración.

`getDescripcion() → "Habitación Estándar"` `getCosto() → 1200 | 1800 | 2800`

DECORADOR ABSTRACTO **ServicioDecorator**

Clase abstracta que implementa `HabitacionComponent` y mantiene una referencia interna al componente que envuelve. Todos los decoradores concretos extienden esta clase, delegan la llamada al componente interior y añaden su propio comportamiento encima.

`protected habitacion: HabitacionComponent`

`abstract getDescripcion(): string` `abstract getCosto(): number`

DECORADORES CONCRETOS **DesayunoDecorator · WifiPremiumDecorator · VistaMarDecorator · JacuzziDecorator**

Cada uno extiende `ServicioDecorator` y agrega su propio costo y texto a la cadena. Al llamar al método, primero delegan al componente envuelto y luego suman su aportación, formando una cadena de responsabilidad.

`getDescripcion() → habitacion.getDescripcion() + ", Desayuno"`

`getCosto() → habitacion.getCosto() + 250`

Flujo de ejecución en HotelService

1. Se instancia el componente base según el tipo recibido en el DTO: `new HabitacionEstandar()`
2. Por cada extra seleccionado, se envuelve la habitación en un nuevo decorador: `habitacion = new DesayunoDecorator(habitacion)`

3. La cadena de decoradores queda anidada, por ejemplo: `WifiPremiumDecorator( DesayunoDecorator( HabitacionEstandar ) )`
4. Al llamar `getCosto()`, cada capa suma su precio al de la capa interior: 1200 + 250 + 120 = **$1,570 MXN**
5. Al llamar `getDescripcion()` se concatenan las descripciones: *"Habitación Estándar, Desayuno, WiFi Premium"*

Ventajas aplicadas en este proyecto

- **Extensibilidad:** Agregar un nuevo servicio (ej. Spa) solo requiere crear un decorador nuevo, sin tocar ninguna clase existente.
- **Composición dinámica:** Las 4 habitaciones × los 4 extras generan 16 combinaciones posibles sin necesitar 16 subclases.
- **Principio Abierto/Cerrado:** Las clases base están cerradas a modificación pero abiertas a extensión mediante decoradores.
- **Transparencia:** El código cliente en `HotelService` solo conoce la interfaz, nunca sabe cuántos decoradores hay apilados.

PATRÓN 02

Bridge

Desacoplar la abstracción de su implementación para que evolucionen independientemente

¿Qué es el patrón Bridge?

El patrón Bridge separa una abstracción (qué hace) de su implementación (cómo lo hace) en dos jerarquías de clases independientes, unidas por una referencia de composición — el "puente". Esto evita la explosión combinatoria de subclases cuando se necesitan variaciones en ambas dimensiones.

Los dos ejes del Bridge

EJE DE ABSTRACCIÓN

¿Cómo se entrega?

Define el canal de entrega del recibo. Puede ser impresión/PDF o correo electrónico. Extiende EntregaRecibo.

EJE DE IMPLEMENTACIÓN

¿Cómo se formatea?

Define cómo se genera el contenido visual del recibo. Actualmente: HTML elegante estilo hotel. Implementa ReciboRenderer.

Diagrama de clases

```
«interface»
ReciboRenderer  (Implementador)
└── + generarHTML(datos: DatosRecibo): string
              △
              │ implements
    PDFReciboRenderer  (Implementador Concreto)
    └── + generarHTML(datos): string
          → retorna HTML completo del recibo con CSS y tipografía del hotel
```

```
EntregaRecibo  (Abstracción)  ←──────────────── ReciboRenderer
├── # renderer: ReciboRenderer                  (referencia al implementador)
└── + entregar(datos): Promise<EntregaResultado>   (abstract)
              △
              │ extends
    ┌─────────┴──────────────┐
    │                        │
ImprimirEntrega          CorreoEntrega
────────────────         ─────────────────────

+ entregar()             - email: string
  1. renderer            + entregar()
     .generarHTML()        1. renderer.generarHTML()
  2. inyecta en iframe   2. emailjs.send({ html_body })
  3. iframe.print()
```

Clases participantes

INTERFACE — IMPLEMENTADOR **ReciboRenderer**

Define la interfaz de bajo nivel para generar el contenido del recibo. Las abstracciones solo conocen esta interfaz, nunca la implementación concreta. Permite sustituir el formato visual (PDF elegante → texto plano → minimalista) sin alterar la lógica de entrega.

`generarHTML(datos: DatosRecibo): string`

IMPLEMENTADOR CONCRETO **PDFReciboRenderer**

Única implementación concreta del renderer. Genera un documento HTML completo con diseño de lujo: tipografía Playfair Display, paleta dorada del Aurum Grand Hotel, desglose de extras, número de reserva y total final. Este HTML se usa tanto para imprimir como para enviar por correo, garantizando que ambos canales muestren exactamente el mismo recibo.

`generarHTML(datos) → string (HTML completo, ~15 KB)`

ABSTRACCIÓN **EntregaRecibo**

Clase abstracta base de las abstracciones refinadas. Recibe el Implementador por inyección en el constructor y lo almacena como referencia protegida. Define el contrato de alto nivel: dado un conjunto de datos de reserva, entrega el recibo por el canal que corresponda.

`constructor(renderer: ReciboRenderer)`

`abstract entregar(datos: DatosRecibo): Promise<EntregaResultado>`

ABSTRACCIÓN REFINADA **ImprimirEntrega**

Implementa la entrega vía impresión/PDF del navegador. Llama al renderer para obtener el HTML, lo inyecta en un `<iframe>` oculto en el DOM y dispara `contentWindow.print()`. Usar un iframe evita los bloqueadores de ventanas emergentes y no interfiere con los estilos de la aplicación principal.

```
entregar(datos) → genera HTML → iframe → window.print()
```

ABSTRACCIÓN REFINADA **CorreoEntrega**

Implementa la entrega vía correo electrónico usando EmailJS. Llama al **mismo renderer** que ImprimirEntrega para obtener idéntico HTML, y lo envía directamente desde el navegador mediante la API de EmailJS sin necesidad de un servidor SMTP propio. Recibe la dirección de correo como parámetro adicional en el constructor.

```
constructor(renderer, email: string)
```

```
entregar(datos) → renderer.generarHTML() → emailjs.send(html_body)
```

Flujo de ejecución en Resumen.tsx

1. El usuario selecciona el canal de entrega: **Imprimir** o **Correo**.
2. Se instancia el Implementador compartido: `const renderer = new PDFReciboRenderer()`
3. Según la opción, se crea la Abstracción correspondiente: `new ImprimirEntrega(renderer)` o `new CorreoEntrega(renderer, email)`
4. Se llama a `entrega.entregar(datosRecibo)`. El método delega al renderer para obtener el HTML.
5. El HTML se procesa según el canal: iframe + print() para impresión, o emailjs.send() para correo.

Ventajas aplicadas en este proyecto

- **Formato único:** ImprimirEntrega y CorreoEntrega producen exactamente el mismo recibo visual porque ambas usan PDFReciboRenderer.
- **Independencia de ejes:** Si se cambia el diseño del recibo (nuevo renderer), ambos canales se actualizan automáticamente sin tocar ImprimirEntrega ni CorreoEntrega.
- **Extensible sin explosión:** Agregar un canal nuevo (ej. WhatsApp) solo requiere crear una nueva abstracción. Agregar un nuevo formato solo requiere un nuevo renderer.
- **Sin dependencia del backend:** CorreoEntrega usa EmailJS directamente desde el navegador, eliminando la necesidad de configuración SMTP en el servidor.

Comparativa: Decorator vs Bridge

ASPECTO	DECORATOR	BRIDGE
Propósito	Agregar responsabilidades dinámicamente a un objeto	Desacoplar abstracción de implementación
Estructura	Cadena de envolturas anidadas sobre un componente	Dos jerarquías unidas por composición
Relación	Decorador implementa la misma interfaz que el componente	Abstracción delega en el implementador
Uso en el proyecto	Calcular precio y descripción de habitación + extras	Entregar el recibo por impresión o correo
Extensión	Nuevo extra → nuevo decorador	Nuevo canal o nuevo formato → nueva clase en su eje
Capa	Backend — NestJS (TypeScript)	Frontend — React + EmailJS (TypeScript)